Saveetha Institute of Medical and Technical Sciences

ASSIGNMENT – 1

CSA02 – C Programming

Grama Jyothi Rural Electricity Co-operative

Analysis and Implementation of an Electricity Billing Program

Student details	Information
Name	Samyuktha.R
Register number	1913260047
Programme	C Programming CSA0209
Submission date	27 October 2026
GitHub repository	https://github.com/samyukth.r360-ui/CSA0209-C-PROGRAMMING-1920260075

Individual Assignment

TASK 1 – Understand the Problem and Identify the Data

The co-operative calculates the electricity bill of each household using the previous and current meter readings. The program has to find the units consumed, calculate the energy charge according to the tariff slabs, add the fixed charge, and display the final bill. The calculation must be accurate even when the meter reading rolls over from 65535 to 0.

1.1 Inputs and outputs

Data item	Why it is needed
Household number	It identifies the household and helps make sure the calculated bill belongs to the correct customer.
Previous reading	It is the reading recorded during the previous billing period.
Current reading	It is used with the previous reading to calculate the consumption for the current period.
Household category	It can be used to choose the correct tariff if different categories have different rates.
Units consumed	It is the amount of electricity used and is needed to calculate the energy charge.
Energy charge	It is calculated using the applicable rates for each consumption slab.
Fixed charge	It is added to the energy charge regardless of the number of units consumed, according to the given tariff.
Total bill	It is the final amount payable and is calculated by adding the energy charge and fixed charge.

1.2 Constants and variables

The tariff values and slab limits should be kept as constants because they do not change while a bill is being calculated. For this assignment, the first 100 units are free, units 101 to 200 cost 225 paise per unit, and units above 200 cost 450 paise per unit. The fixed charge is 5000 paise, which is ₹50. Keeping these values in one place makes it easier to revise the tariff without missing a calculation.

The household number, meter readings, units consumed and calculated bill are variables because their values can be different for each household. The number of households being processed can also change.

1.3 Values that may be negative, fractional or large

Meter readings and valid electricity consumption are non-negative. However, subtracting a larger previous reading from a smaller current reading can produce a negative intermediate result when the meter rolls over. The program must handle this case correctly. Tariff amounts may contain paise, so storing money as integer paise is more reliable than storing rupees in a floating-point type. The program may process up to 800,000 households, so the type used for total consumption or total revenue must be large enough for the accumulated value.

1.4 Rules for a correct bill

1. The units consumed must be calculated correctly, including a meter rollover.
2. The first 100 units are charged at 0 paise per unit.
3. Units from 101 to 200 are charged at 225 paise per unit.

4. Every unit above 200 is charged at 450 paise per unit.
5. The fixed charge of ₹50 is added to the energy charge.
6. The final bill must not be negative for valid input.
7. The total bill must equal the energy charge plus the fixed charge.
8. The same tariff values must be used consistently for all households that belong to the same tariff category.

For example, if a household uses 150 units, the first 100 units are free and the next 50 units cost 50 × ₹2.25 = ₹112.50. After adding the fixed charge of ₹50, the total bill is ₹162.50.

1.5 Assumptions

- Each reading is a valid value between 0 and 65535.
- The meter is a 16-bit counter and rolls over from 65535 to 0.
- The sample tariff and fixed charge given in the question are used for the program.
- Only one rollover can occur between two readings. If multiple rollovers are possible, extra information is needed to calculate the consumption correctly.
- Different household categories may use different tariffs in a real system, but the sample program uses the rates given in the assignment.

1.6 Alternative: storing readings as strings

The readings could also be stored as character strings, for example "01530" and "01860". In that case, the program would first convert the strings into integers using a function such as strtoul(), or perform arithmetic digit by digit. If d is the number of digits, converting a string or comparing two strings can take O(d) time. A fixed-width integer reading can be compared and used in arithmetic in O(1) time.

Point	Integer	Character string
Example	1530	"01530"
Storage	A fixed number of bytes for the chosen integer type	d + 1 bytes for d digits and the null terminator
Conversion	Not needed	O(d) to convert
Comparison	O(1) for a fixed-width integer	Up to O(d)
Leading zeroes	Not preserved	Preserved

For this problem, integer storage is simpler because the readings are numbers that must be subtracted and checked for rollover.

TASK 2 – Choose the Data Types

I would choose the data type based on the possible values stored in each variable. The exact size of each type depends on the compiler and system, so I would use sizeof and the limits in limits.h and float.h to check my choices.

Data item	Suggested type	Reason
Previous and current readings	unsigned short	Suitable if it is 2 bytes and its maximum value is 65535 on the target system.
Household number	unsigned int	Used for a non-negative identifier, provided the maximum identifier fits.
Household category	enum	Useful for a fixed set of categories, such as domestic and commercial.
Units consumed	unsigned int	Consumption is non-negative; the range should be checked for the expected meter rules.
Tariff rates and fixed charge	long	Stores whole paise values.
Energy charge and one bill	long	Stores money in paise without fractional floating-point values, provided overflow is avoided.
Total revenue	long long	Provides a larger range for adding many household bills.

2.1 Program to check sizes and limits

Save this program as Task02/type_info.c.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("Size of char: %zu bytes\n", sizeof(char));
    printf("Size of short: %zu bytes\n", sizeof(short));
    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of long: %zu bytes\n", sizeof(long));
    printf("Size of long long: %zu bytes\n", sizeof(long long));
    printf("Size of float: %zu bytes\n", sizeof(float));
    printf("Size of double: %zu bytes\n", sizeof(double));

    printf("\nInteger limits:\n");
    printf("CHAR_MIN = %d\n", CHAR_MIN);
    printf("CHAR_MAX = %d\n", CHAR_MAX);
    printf("SHRT_MIN = %d\n", SHRT_MIN);
    printf("SHRT_MAX = %d\n", SHRT_MAX);
    printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MIN = %d\n", INT_MIN);
    printf("INT_MAX = %d\n", INT_MAX);
    printf("UINT_MAX = %u\n", UINT_MAX);
    printf("LONG_MIN = %ld\n", LONG_MIN);
    printf("LONG_MAX = %ld\n", LONG_MAX);
    printf("LLONG_MIN = %lld\n", LLONG_MIN);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);

    printf("\nFloating-point limits:\n");
    printf("FLT_MIN = %e\n", FLT_MIN);
    printf("FLT_MAX = %e\n", FLT_MAX);
    printf("DBL_MIN = %e\n", DBL_MIN);
    printf("DBL_MAX = %e\n", DBL_MAX);
    return 0;
}
```

```
12      printf("Size of float: %zu bytes\n"
13      printf("Size of double: %zu bytes\n
14
```

```
Size of char: 1 bytes
Size of short: 2 bytes
Size of int: 4 bytes
Size of long: 8 bytes
Size of long long: 8 bytes
Size of float: 4 bytes
Size of double: 8 bytes

Integer limits:
CHAR_MIN = -128
CHAR_MAX = 127
SHRT_MIN = -32768
SHRT_MAX = 32767
USHRT_MAX = 65535
INT_MIN = -2147483648
INT_MAX = 2147483647
UINT_MAX = 4294967295
LONG_MIN = -9223372036854775808
LONG_MAX = 9223372036854775807
LLONG_MIN = -9223372036854775808
LLONG_MAX = 9223372036854775807

Floating-point limits:
FLT_MIN = 1.175494e-38
FLT_MAX = 3.402823e+38
DBL_MIN = 2.225074e-308
DBL_MAX = 1.797693e+308


...Program finished with exit code 0
Press ENTER to exit console.
```

The output shows the size and range of the types on my system. I should not assume that long has the same size on every platform. FLT_MIN and DBL_MIN are the smallest positive normalised values, not the most negative values.

2.2 Why I would store money in paise

I would not store money as float rupees because many decimal fractions cannot be represented exactly in binary floating-point. Small errors can appear when many bills are added. Instead, I would store money as an integer number of paise. For example, ₹0.10 is stored as 10 paise and ₹2.25 is stored as 225 paise. This gives exact whole-paise arithmetic as long as the integer type does not overflow.

2.3 Floating-point experiment for 4,000 bills

Save this as Task02/float_error.c. It adds ₹0.10 four thousand times using float and compares it with integer paise.

```
#include <stdio.h>

int main(void)
{
    float total = 0.0f;
    long exact_total = 0;

    for (int i = 0; i < 4000; i++) {
        total += 0.10f;
        exact_total += 10;
    }

    printf("Float total: Rs %.8f\n", total);
    printf("Integer total: Rs %ld.%02ld\n",
           exact_total / 100, exact_total % 100);
    return 0;
}
```

Compile and run:

```c
#include <stdio.h>

int main(void)
{
    float total = 0.0f;
    long exact_total = 0;

    for (int i = 0; i < 4000; i++) {
        total += 0.10f;
        exact_total += 10;
    }

    printf("Float total: Rs %.8f\n", total);
    printf("Integer total: Rs %ld.%02ld\n",
           exact_total / 100, exact_total % 100);
    return 0;
}
```

```
Float total: Rs 400.01519775
Integer total: Rs 400.00


...Program finished with exit code 0
Press ENTER to exit console.
```

The mathematical total is ₹400.00. The integer-paise result should be exactly ₹400.00. I will record the float result shown by my own compiler and include a screenshot as evidence.

2.4 Complete the code: meter rollover

The meter is a 16-bit counter. I can handle one rollover using unsigned 16-bit arithmetic without writing an if–else statement.

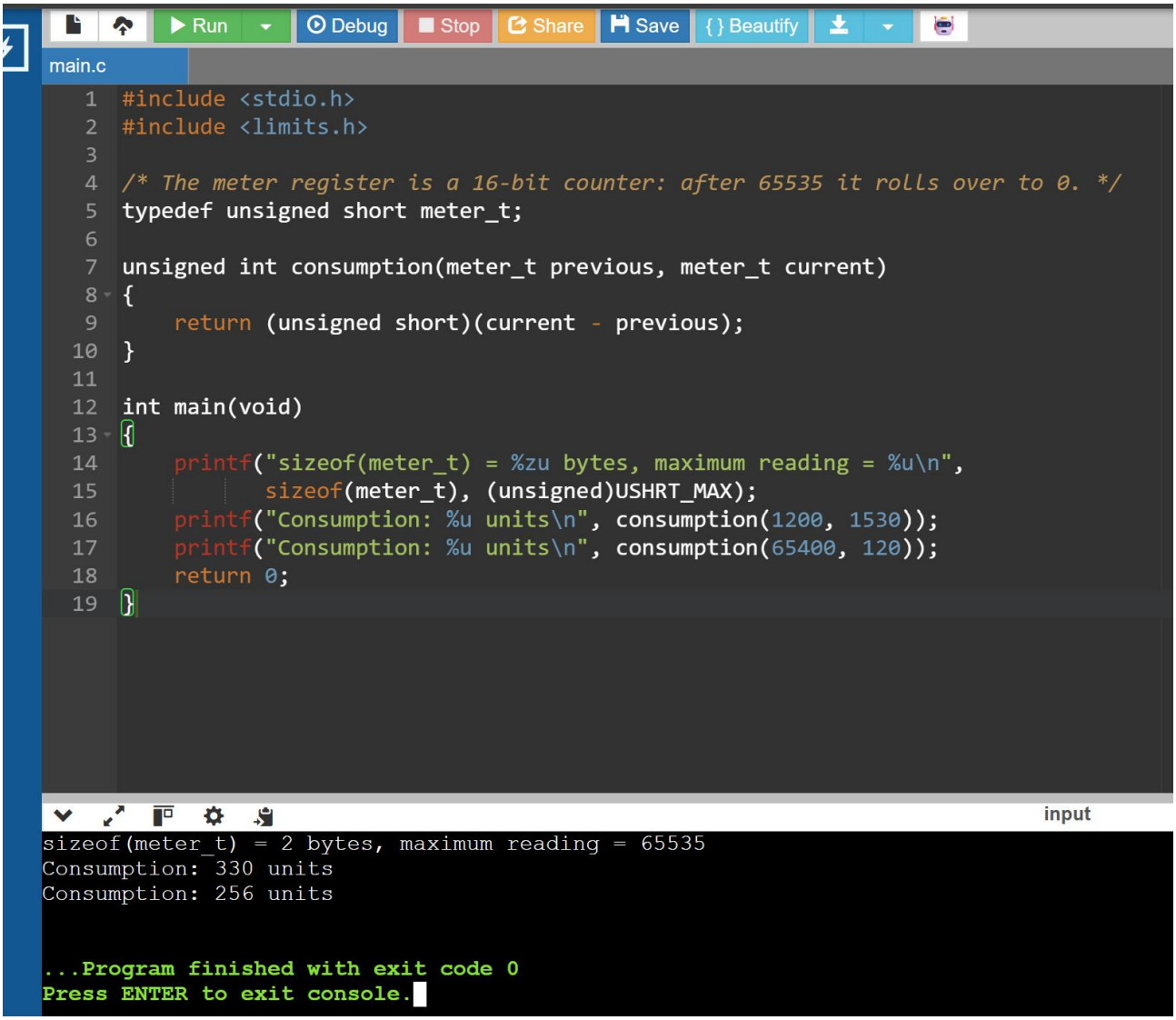
```c
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
```

```
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```



The second result is 256 because the meter wraps around at 65536. The calculation is (120 − 65400) modulo 65536, which gives 256. Casting the subtraction result to unsigned short wraps it into the unsigned 16-bit range. C does not guarantee that unsigned short is 16 bits on every system, so the size and maximum must be checked.

2.5 Alternative: using double rupees

If I used double rupees, I could calculate the bill in rupees and round it to two decimal places when converting it to the payable amount. However, decimal values such as 0.10 may still be approximated internally. Integer paise is exact for whole-paise amounts. Both approaches take O(1) time per addition and O(n) time to add n bills, with O(1) extra space for a running total. The main difference is monetary precision, not Big-O time.

Feature	Integer paise	Double rupees
Time per addition	O(1)	O(1)
Typical storage	Depends on selected integer type	Usually 8 bytes
Precision	Exact to whole paise if no overflow	Some decimal fractions are approximate
Rounding	Usually not needed during whole-paise calculations	Needed at a defined billing boundary
Total for n bills	O(n) time, O(1) extra space	O(n) time, O(1) extra space

TASK 3 – Define the Constants

I would define the slab limits and tariff rates once at the top of the program. This makes the code easier to maintain because a tariff revision can be made in one place.

3.1 Difference between #define, const and enum

Feature	#define	const	enum
How it works	Text substitution by preprocessor	Read-only object with a declared type	Named integral constants
Type checking	No inherent type	Yes	Integer-valued constants
Scope	Preprocessor definition, not block-scoped	Normal C scope	Depends on declaration context
Debugger	No object for the macro itself	May be visible	Names may be visible
Storage	Macro itself uses no storage	Object may use storage	Enumeration constants use no storage

For the supplied tariff, #define is suitable because the values are simple compile-time constants. I would use enum for a fixed set of categories, and const when I need a typed read-only object.

3.2 Completed billing program

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }

    if (units > SLAB1_UNITS) {
        int slab2_units = units < SLAB2_UNITS
                          ? units - SLAB1_UNITS
                          : SLAB2_UNITS - SLAB1_UNITS;
        paise += (long)slab2_units * RATE2_PAISE;
    }

    if (units > 0) {
        int slab1_units = units < SLAB1_UNITS
                          ? units
                          : SLAB1_UNITS;
        paise += (long)slab1_units * RATE1_PAISE;
    }

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

main.c

```
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }

    if (units > SLAB1_UNITS) {
```

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00


...Program finished with exit code 0
Press ENTER to exit console.
```

3.3 Testing a tariff revision

I would change RATE2_PAISE from 225 to 230, recompile the program, and run it again. The expected revised bills are:

Units	Original bill	Bill after changing rate to 230
0	₹50.00	₹50.00
100	₹50.00	₹50.00
150	₹162.50	₹165.00
200	₹275.00	₹280.00
350	₹950.00	₹955.00

The first slab is still free and the third-slab rate is unchanged. Only the units charged at the second-slab rate are affected. The bill for 350 units increases by ₹5 because 100 units are charged an extra ₹0.05 per unit.

3.4 Alternative concept and complexity

If I used const variables instead of #define, each value would have a declared type and normal C scope. The compiler and debugger may also handle these values more clearly. An enum is useful for integral options such as household categories, but is not the best choice for every monetary rate. The choice does not meaningfully change the running time. Each household's bill takes O(1) time because the number of slab calculations is fixed; processing n households takes O(n) time.

TASK 4 – Use Macros Safely

Macros are expanded by the preprocessor before the compiler processes the program. They are useful for small repeated expressions, but the arguments and the complete expression should be surrounded by parentheses.

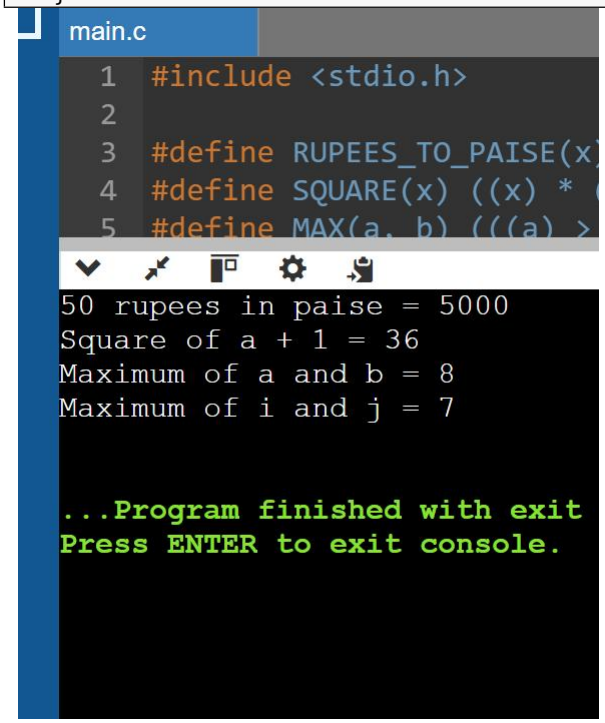
4.1 Program using macros

```c
#include <stdio.h>

#define RUPEES_TO_PAISE(x) ((x) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 5;
    int b = 8;
    int i = 3;
    int j = 7;

    printf("50 rupees in paise = %d\n", RUPEES_TO_PAISE(50));
    printf("Square of a + 1 = %d\n", SQUARE(a + 1));
    printf("Maximum of a and b = %d\n", MAX(a, b));
    printf("Maximum of i and j = %d\n", MAX(i, j));
    return 0;
}
```



4.2 Why SQUARE needs parentheses

If I write #define SQUARE(x) x * x, then SQUARE(a + 1) expands to a + 1 * a + 1. When a is 5, this gives 11 because multiplication has higher precedence than addition. The safer definition #define SQUARE(x) ((x) * (x)) expands to ((a + 1) * (a + 1)), which gives 36.

4.3 Why MAX can evaluate an argument twice

The macro MAX(a, b) compares the arguments and then evaluates the selected argument again. If I call MAX(i++, j), i++ may be evaluated during the comparison and again if the true branch is selected. This can increment i twice. Therefore, I should avoid passing expressions with side effects to such macros.

4.5 Alternative: static inline functions

```
static inline int square_int(int x)
{
    return x * x;
}

static inline int max_int(int a, int b)
{
    return a > b ? a : b;
}
```

A function evaluates each argument expression once and provides normal type checking. The compiler may inline the function, although this is not guaranteed. Both a macro and a function for these small integer operations take O(1) time. If a macro is used k times, its expanded code may be repeated at those locations. A shared function may reduce duplication, while an inlined function can also be duplicated by the compiler. Actual code size and speed depend on compiler optimisation.

Point	Macro	static inline function
Time for square/max	O(1)	O(1)
Argument evaluation	May happen more than once	Each argument expression is evaluated once
Type checking	No normal function parameter checking	Declared parameter types are checked
Code size	Expansion may repeat at k uses	May share code or be inlined
Debugging	Can be harder	Usually easier

Design Challenge – Smart Meters and Time-of-Day Tariffs

I would modify the original design instead of keeping it unchanged. The integer-paise approach is still useful for money, but smart meters report readings every 15 minutes and the rate may depend on the time of day. The program would need to store the timestamp and consumption for each interval, identify the tariff period for that timestamp, calculate the interval charge, and add the charges for each household. It should also validate missing or duplicate readings and use sufficiently large types for cumulative usage and revenue. The tariff rules should be stored in a clear table or configuration rather than repeated throughout the code.

Therefore, I would retain the basic ideas of integer paise and centralised constants, but modify the data model and tariff calculation to support interval-based readings.

Conclusion

From this assignment, I identified the data required for electricity billing and selected types based on the values they need to store. Integer paise helps avoid floating-point errors in monetary calculations, and unsigned arithmetic can handle a single meter rollover when the counter width is known. Keeping tariff values in one place makes revisions safer. Parenthesised macros prevent precedence errors, although inline functions are safer when arguments may have side effects. For smart meters, the design would need to be extended to process timestamped readings and time-of-day tariffs.